NVFP4-DiT: Efficient 4-Bit Audio-Guided Video Diffusion Transformers for Low-Cost Video Generation

Md Rakibul Islam Raihan, Peng Zhang

***Abstract*—Audio-guided video diffusion transformers produce high-quality visual content, but their training and inference costs remain a major barrier to practical deployment. We present NVFP4-DiT, a low-precision framework for audio-guided text-to-video generation in 4-bit floating-point precision using NVIDIA's NVFP4 format. The framework combines a theoretical analysis of FP4 diffusion under block-wise scaling, including a sufficient condition for preserving temporal coherence, with an adaptive block-wise quantization strategy and quantization-aware training (QAT) using learnable scales for cross-modal attention. To support efficient video generation, we further describe custom Triton kernels for FP4-packed matrix multiplication and attention. Experiments on WebVid-10M, VGGSound, and UCF-101 evaluate generation quality, synchronization, latency, throughput, and memory use across multiple DiT variants. The results indicate that NVFP4-DiT preserves competitive video quality and audio-visual alignment while substantially reducing memory footprint and computational cost.**

***Index Terms*—Diffusion Transformers, Low-Precision Training, NVFP4, Multimodal Video Generation, Quantization-Aware Training, CUDA Kernels.**

I. Introduction

A. Motivation

Diffusion transformers (DiTs) [1] have become a powerful foundation for modern video generation, enabling high-fidelity synthesis from text and audio prompts [2]–[4]. Recent systems such as Sora [5], Stable Video Diffusion [6], and ModelScope-T2V [7] demonstrate strong capability in producing temporally coherent and semantically aligned video content. However, the computational cost of these models remains prohibitive: training a single DiT can require hundreds to thousands of GPU days, while inference can still demand several seconds per generated frame on high-end accelerators [8], [9].

This cost barrier limits the use of generative video models in practical video systems, including interactive media tools, low-latency content creation, and resource-constrained deployment. Although model compression techniques such as pruning [10], distillation [11], and quantization [12] have shown promise in image generation, video diffusion introduces additional challenges: temporal coherence must be preserved across frames, audio and visual streams must remain synchronized, and spatiotemporal attention incurs high computational complexity.

Md Rakibul Islam Raihan is with the School of Software, Northwestern Polytechnical University, Xi'an, China (e-mail: raihan@mail.nwpu.cn).

Peng Zhang is with the School of Computer Science, Northwestern Polytechnical University, Xi'an, China (e-mail: zh0036ng@nwpu.edu.cn).

B. Low-Precision Opportunities

NVIDIA's NVFP4 format [13]—a 4-bit floating-point representation with 1 sign bit, 2 exponent bits, and 1 mantissa bit—offers unprecedented compression for deep neural networks. When combined with tensor core acceleration on H100 and B200 GPUs, FP4 can theoretically achieve $4\times$ memory reduction and $3\times$ throughput improvement over FP16 [14]. However, three critical questions remain unanswered:

1) **Theoretical:** Can diffusion models—which are notoriously sensitive to numerical precision—converge and generate coherent videos under extreme 4-bit quantization?
2) **Algorithmic:** How should quantization be adapted for the spatiotemporal dynamics of video diffusion, where temporal consistency demands precise attention patterns across frames?
3) **Systems:** What kernel-level optimizations are necessary to realize the theoretical gains of FP4 for video generation workloads?

C. Contributions

This paper addresses these questions through **NVFP4-DiT**, a framework for efficient 4-bit training and inference of audio-guided video diffusion transformers. Our contributions are:

1) **Theoretical Analysis (Section III):** We derive convergence bounds for FP4-trained diffusion models under block-wise scaling and provide a sufficient condition for preserving temporal coherence, namely $\|\Delta Q K^T\|_F \leq \delta_t \cdot \|\mu\|_2$ for temporal difference δ_t and mean attention μ.
2) **Algorithmic Design (Section IV):** We introduce **adaptive block-wise quantization** for spatiotemporal diffusion layers and **quantization-aware training (QAT)** with learnable scale factors for cross-modal attention between audio spectrograms and visual latents.
3) **Systems Optimization (Section V):** We develop custom Triton [15] kernels for FP4-packed matrix multiplication and attention, and we describe how the implementation integrates with vLLM [16] and TensorRT-LLM [17] for efficient batched inference.
4) **Empirical Evaluation (Section VI):** We evaluate NVFP4-DiT on WebVid-10M [18], VGGSound [19], and UCF-101 [20], showing competitive quality relative to BF16 together with substantial reductions in memory use, throughput cost, and training energy.

D. Paper Organization

Section II reviews related work. Section III presents theoretical foundations. Section IV describes our algorithmic framework. Section V details systems optimizations. Section VI reports experimental results. Section VII ablates design choices. Section VIII discusses limitations and impact. Section IX concludes.

II. Related Work

A. Video Diffusion Models

Diffusion models [21], [22] generate data by reversing a gradual noising process. For video, early approaches extended image diffusion with temporal layers [23], [24]. Recent transformer-based architectures [1], [25] unify spatial and temporal attention in a single DiT backbone, achieving state-of-the-art results. Audio-guided variants [26], [27] condition on mel-spectrograms via cross-attention, enabling synchronized audiovisual generation. However, all these models operate at FP16/BF16 precision, incurring substantial computational costs.

B. Quantization for Generative Models

Quantization reduces precision to compress models and accelerate inference. Post-training quantization (PTQ) [28], [29] is applied after training but can degrade quality in generative models. Quantization-aware training (QAT) [30], [31] simulates quantization during training and is generally more robust at low bitwidths. Prior work on image diffusion has shown that low-precision deployment is feasible in less demanding settings [32]. For video, however, low-bit quantization remains challenging because temporal coherence must be preserved across frames. Our work studies how 4-bit training can be stabilized in this setting.

C. Low-Precision Formats

NVIDIA's FP8 [33] and NVFP4 [13] formats enable extreme compression. FP8 (E4M3/E5M2) has been applied to LLM training [34] and inference [35]. NVFP4—with 2 exponent bits and 1 mantissa bit—provides dynamic range up to $2^{2^2} = 16$ and precision of $2^{-1} = 0.5$ relative to scale. Prior work on FP4 has primarily focused on language and vision backbones [36]. Here we study the additional challenges that arise in video diffusion, including temporal coherence and cross-modal synchronization.

D. Systems for Efficient Inference

vLLM [16] introduced PagedAttention and continuous batching for LLM serving. TensorRT-LLM [17] provides optimized kernels and graph optimizations for transformer inference. In this work, we draw on design ideas from both systems and discuss how similar serving strategies can be adapted to spatiotemporal video diffusion workloads.

III. Theoretical Foundations

A. Preliminaries

Let $x_0 \in \mathbb{R}^{C\times F\times H\times W}$ represent a video tensor with C channels, F frames, height H, and width W. The forward diffusion process adds Gaussian noise over T timesteps:

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t I) \tag{1}$$

where $\beta_t \in (0,1)$ is a variance schedule. The reverse process learns a denoising network $\epsilon_\theta(x_t, t, c)$ conditioned on text c_{text} and audio c_{audio}:

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t, c), \sigma_t^2 I) \tag{2}$$

B. FP4 Quantization Error Bounds

Definition III.1 (NVFP4 Quantization)**.** For a floating-point value v with scale factor $s \in \mathbb{R}^+$, the NVFP4 quantized value is:

$$Q_{\text{FP4}}(v; s) = s \cdot \text{clip}(\lfloor v/s \rceil_{\text{FP4}}, -6, 6) \tag{3}$$

where $\lfloor\cdot\rceil_{\text{FP4}}$ rounds to the nearest representable FP4 value, and clipping limits the exponent range to $[-6, 6]$ in the scaled domain.

Lemma III.2 (Quantization Error Bound)**.** *For any $v \in \mathbb{R}$, the relative quantization error satisfies:*

$$\left|\frac{Q_{FP4}(v; s) - v}{v}\right| \leq \epsilon_{FP4} = 2^{-3} = 0.125 \tag{4}$$

Proof. FP4 has 1 mantissa bit, giving a relative precision of $2^{-1} = 0.5$ in the representable range. With optimal scaling, the maximum relative error is half of this, yielding 0.25. However, block-wise scaling with 2×2 blocks reduces error to 0.125. The complete proof follows from [13, Theorem 1]. □

Theorem III.3 (Convergence of FP4-Diffusion)**.** *Let ϵ_{FP} be the denoising network trained with FP4 quantization. Under Assumptions 1-3 (Lipschitz continuity of ϵ_θ, bounded gradients, and smooth noise schedule), the reverse process error satisfies:*

$$\mathbb{E}[\|x_0 - \hat{x}_0\|_2^2] \leq \frac{C}{T}\sum_{t=1}^{T}\left(\epsilon_{FP4}^2 \cdot \mathbb{E}[\|\nabla_{x_t}\log p(x_t)\|_2^2] + \sigma_t^2\right) \tag{5}$$

where C is a constant depending on model architecture and T is the number of diffusion steps.

Proof Sketch. The proof extends the convergence analysis of DDIM [22] to account for quantization error. At each step, the FP4-quantized prediction introduces an error term bounded by $\epsilon_{\text{FP4}}\|\nabla \log p(x_t)\|$. Accumulating this term over T steps and applying the Lipschitz property yields the stated bound. A complete derivation is provided in Appendix A. □

Corollary III.4. *For sufficiently large T ($T \geq 1000$), the FP4 convergence error approaches the FP16 baseline up to a multiplicative factor of $(1 + O(\epsilon_{FP4}))$.*

C. Temporal Coherence Under Quantization

Definition III.5 (Temporal Coherence)**.** A generated video sequence $\{x^{(f)}\}_{f=1}^{F}$ exhibits temporal coherence if the frame-to-frame difference satisfies:

$$\|x^{(f+1)} - x^{(f)}\|_F \leq \tau \cdot \|x^{(f)}\|_F \quad \forall f \tag{6}$$

for some $\tau \ll 1$.

Theorem III.6 (Coherence Preservation Condition)**.** *Let $A = QK^T$ be the attention matrix in a DiT block. Under FP4 quantization, the temporal coherence error Δ_{temp} satisfies:*

$$\Delta_{temp} \leq \frac{2\epsilon_{FP4}}{1 - \epsilon_{FP4}} \cdot \|\mu\|_2 + \delta_t \tag{7}$$

where μ is the mean attention across frames and $\delta_t = \|A_{FP16}^{(f+1)} - A_{FP16}^{(f)}\|_F$ is the baseline temporal difference. A sufficient condition for coherence preservation is:

$$\|\Delta Q K^T\|_F \leq \delta_t \cdot \|\mu\|_2 \tag{8}$$

This theorem guides our adaptive quantization strategy: allocate higher precision (smaller ϵ) to attention heads with large temporal gradients.

IV. Algorithmic Framework

A. Adaptive Block-Wise Quantization

Standard block-wise quantization [37] divides tensors into fixed-size blocks, each with its own scale factor. For video diffusion, we observe that spatial and temporal dimensions have different sensitivity to quantization.

Spatial sensitivity arises from fine-grained textures and edges. **Temporal sensitivity** arises from motion continuity and object persistence.

Our **adaptive block-wise quantization** dynamically determines block size per layer:

$$\text{BlockSize}(l) = \text{argmin}_{b \in \mathcal{B}} \left(\text{Error}(l, b) + \lambda \cdot \frac{\text{Memory}(b)}{\text{Memory}_{\max}} \right) \tag{9}$$

where $\mathcal{B} = \{1, 2, 4, 8, 16, 32, 64\}$ (block sizes along spatial dimension), $\text{Error}(l, b)$ is measured on a calibration set, and λ balances accuracy and memory.

Algorithm 1 summarizes the adaptive selection procedure.

Algorithm 1 Adaptive Block-Wise Quantization

Require: Layer l, calibration set D_{cal}, candidate blocks $\mathcal{B}$, trade-off λ
Ensure: Optimal block size b^*
for each $b \in \mathcal{B}$ **do**
  Quantize layer l with block size b
  Compute reconstruction error err[b]
  Compute memory ratio mem[b]
  Compute cost: cost[b] = err[b] + $\lambda \cdot$ mem[b]
end for
$b^* = \text{argmin}_b \text{cost}[b]$
return b^*

B. Quantization-Aware Training for Cross-Modal Attention

Cross-modal attention between visual latents z_{vis} and audio features z_{audio} is critical for synchronization. We propose **learnable scale factors** per attention head:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + \log(s_{\text{head}}) \right) V \tag{10}$$

where $s_{\text{head}} \in \mathbb{R}^H$ are learnable scale factors initialized to 1.

The FP4 quantization simulation during training follows a straight-through estimator approach: during the forward pass, we quantize to FP4 and dequantize; during the backward pass, gradients bypass the quantization operation using the identity function. This enables the model to learn parameters that are robust to quantization error.

Algorithm 2 details the QAT procedure.

Algorithm 2 Quantization-Aware Training for NVFP4-DiT

Require: Pretrained BF16 model M, training data $\mathcal{D}$, learning rate η
Ensure: FP4-quantized model M_q
Initialize M_q with weights from M
Insert fake quantization ops (Algorithm 1) after each linear and attention layer
for epoch = 1 to E **do**
  for batch in $\mathcal{D}$ **do**
    # Forward pass with simulated FP4
    pred = M_q(batch)
    loss = $\mathcal{L}_{\text{diffusion}}$(pred, batch) + $\lambda_{\text{sync}} \mathcal{L}_{\text{sync}}$(pred, batch)
    # Backward pass with straight-through estimator
    loss.backward()
    for each learnable scale factor s **do**
      Apply gradient scaling for stable updates
    end for
    optimizer.step()
  end for
end for
return M_q

C. Loss Functions

We optimize a combined loss:

$$\begin{aligned} \mathcal{L}_{\text{total}} = & \underbrace{\mathbb{E}_{t,\epsilon,x_0} \|\epsilon - \epsilon_\theta(x_t, t, c)\|_2^2}_{\text{Diffusion loss}} \\ & + \lambda_{\text{sync}} \underbrace{\text{SyncNet}(x_{\text{frames}}, c_{\text{audio}})}_{\text{Synchronization loss}} \\ & + \lambda_{\text{temp}} \underbrace{\|x_t - x_{t-1}\|_F^2}_{\text{Temporal smoothness}} \end{aligned} \tag{11}$$

where SyncNet [38] computes lip-sync error between generated frames and audio.

V. Systems Optimization

A. FP4-Packed Triton Kernels

Naive FP4 implementation suffers from three main inefficiencies: (1) memory overhead from unpacking FP4 to FP16 for computation, (2) low tensor core utilization for non-standard data types, and (3) poor cache locality for block-wise scales.

Algorithm 3 Continuous Batching for Video Diffusion

Require: Request queue $\mathcal{Q}$, GPU memory budget M
Ensure: Generated videos
Initialize frame_kv_cache = {}
while $\mathcal{Q}$ not empty **do**
  # Dynamic batching
  batch = []
  for each request req in $\mathcal{Q}$ **do**
    if memory_estimate(req) + current_memory $< M$ **then**
      batch.append(req)
    end if
  end for
  # Prepare batch tensors (pad to max frames in batch)
  batch_padded = pad_to_max_frames(batch)
  # Run one diffusion denoising step
  pred = model(batch_padded, frame_kv_cache)
  # Update cache asynchronously
  frame_kv_cache.update(pred)
  # Remove finished requests from queue
  $\mathcal{Q} = \{ req \in \mathcal{Q} \mid \neg req.\text{done} \}$
end while

Our Triton kernels address these challenges through three key innovations. First, we pack two FP4 values into a single byte, reducing memory bandwidth by $4\times$ compared to FP16. Second, we perform on-the-fly dequantization directly within tensor core operations, avoiding intermediate FP16 expansion. Third, we optimize block-wise scale loading with vectorized memory accesses to maximize cache locality.

The kernel operates as follows: given input matrices stored in packed FP4 format with per-block scale factors, the kernel loads packed bytes, unpacks them to FP16 using the corresponding scales, performs matrix multiplication using Tensor Cores, and accumulates results in FP32. This design achieves:

- $2.1\times$ speedup vs. naive FP4 (unpack $\rightarrow$ FP16 $\rightarrow$ compute)
- $3.2\times$ speedup vs. FP16 baseline
- $4\times$ memory reduction for weights and activations

B. vLLM Integration for Video Diffusion

We adapt vLLM's continuous batching for video generation. The key challenge is that video diffusion has variable-length generation (different numbers of frames per request) and variable compute per step (attention complexity scales quadratically with frame count F).

Our solution implements three components:

Frame-level paged attention: The key-value cache is partitioned into fixed-size pages at the frame granularity. Each page stores attention states for a contiguous block of frames. This enables efficient memory sharing across requests and reduces fragmentation.

Dynamic frame batching: Requests are grouped by their remaining frame count. Batches are formed dynamically before each diffusion step, selecting requests with similar remaining frames to minimize padding overhead.

Speculative frame decoding: Multiple future frames are generated in parallel using a lightweight predictor, then verified by the full model with rejection sampling. This trades marginal compute for significant latency reduction when predictions are accurate.

Algorithm 3 shows our inference serving scheduler.

VI. Experiments

A. Experimental Setup

Datasets:

TABLE I
Dataset Statistics

Dataset	Type	Size	Resolution
WebVid-10M	Text-Video	10M clips	336x336
VGGSound	Audio-Video	200k clips	224x224
UCF-101	Action	13k videos	320x240

Models:

- DiT-S (small), DiT-B (base), DiT-L (large) [1]
- Stable Video Diffusion (SVD) [6]
- ModelScope-T2V [7]

Hardware:

- $8\times$ NVIDIA H100 (80GB) for training
- $1\times$ H100 for inference benchmarks

Baselines:

- BF16 (full precision)
- FP8 (E4M3) with QAT [33]
- INT8 with SmoothQuant [29]
- FP4 with static quantization

Metrics:

- **FVD** (Fréchet Video Distance): lower is better
- **SyncNet** accuracy: lip-audio sync (0-1)
- **CLIP-T** score: text-video alignment
- **LPIPS**: temporal consistency (lower is better)
- Throughput (videos/second), Latency (ms/frame), Memory (GB), Energy (J/video)

B. Main Results

TABLE II
Quality Comparison on WebVid-10M Test Set

Method	Precision	FVD	SyncNet	CLIP-T	LPIPS
DiT-B (baseline)	BF16	98.3	0.892	0.312	0.087
DiT-B + FP8	FP8	100.1	0.885	0.308	0.089
DiT-B + INT8	INT8	105.7	0.871	0.301	0.094
DiT-B + static FP4	FP4	187.3	0.723	0.241	0.156
NVFP4-DiT (ours)	FP4	99.2	0.886	0.309	0.088

Key observations:

- NVFP4-DiT achieves 0.9% FVD degradation vs. BF16 (99.2 vs. 98.3)
- SyncNet accuracy drops only 0.7% (0.886 vs. 0.892)
- Static FP4 collapses completely (FVD 187.3)

TABLE III
Quality Across Model Architectures

Model	Precision	FVD	SyncNet	Memory (GB)
DiT-S	BF16	112.4	0.854	14.2
DiT-S	NVFP4-DiT	113.8	0.849	3.2
DiT-B	BF16	98.3	0.892	28.5
DiT-B	NVFP4-DiT	99.2	0.886	6.8
DiT-L	BF16	87.6	0.917	56.3
DiT-L	NVFP4-DiT	88.9	0.912	13.5
SVD	BF16	82.1	0.903	42.1
SVD	NVFP4-DiT	83.4	0.898	10.1

Memory reduction: 76.3% on average.

TABLE IV
INFERENCE PERFORMANCE (BATCH SIZE=4, 32 FRAMES)

Method	Throughput (videos/s)	Latency (ms/frame)	Memory (GB)	Energy (J/video)
BF16 baseline	0.31	101.2	28.5	892
FP8	0.67	46.8	14.3	412
INT8	0.72	43.6	12.1	384
NVFP4-DiT (ours)	1.05	29.8	6.8	285

Improvements vs. BF16:

- Throughput: 3.4× (1.05 vs. 0.31 vid/s)
- Latency: 3.4× reduction (29.8 vs. 101.2 ms/frame)
- Memory: 76.3% reduction (6.8 vs. 28.5 GB)
- Energy: 68% reduction (285 vs. 892 J/video)

C. Audio-Visual Synchronization Results

TABLE V
SYNCNET ACCURACY ON VGGSOUND UNDER NOISE CONDITIONS

Condition	BF16	NVFP4-DiT	Difference
Clean audio	0.892	0.886	-0.006
+ 10dB noise	0.845	0.839	-0.006
+ 20dB noise	0.781	0.774	-0.007
Cross-modal (different audio)	0.023	0.021	-0.002

Finding: NVFP4-DiT preserves synchronization accuracy across noise conditions, with degradation consistently below 0.7%.

D. Training Efficiency

TABLE VI
TRAINING COST COMPARISON (DIT-B ON WEBVID-10M)

Precision	GPU hours	Energy (MWh)	Cost ($)
BF16	4,800	28.8	9,600
FP8	2,400	14.4	4,800
NVFP4-DiT	1,600	9.6	3,200

Reduction: 66.7% GPU hours, 66.7% energy, 66.7% cost. These gains suggest that the proposed low-precision training pipeline can reduce end-to-end computational expense while preserving the quality trends reported in the preceding results. In practical deployment settings, this reduction can translate to shorter retraining cycles, lower serving cost per generated clip, and improved accessibility for iterative model development.

VII. ABLATION STUDIES

A. Block Size Analysis

Table VII summarizes how quantization block size affects generation quality.

TABLE VII
EFFECT OF BLOCK SIZE ON FVD (DIT-B, WEBVID-10M)

Block Size	FVD	SyncNet
1x1 (per-element)	98.7	0.888
4x4	98.9	0.887
8x8	99.2	0.886
16x16	100.1	0.882
32x32	104.3	0.871
Adaptive (ours)	**98.9**	**0.888**

B. QAT Ablation

Table VIII isolates the contribution of each quantization-aware training component.

TABLE VIII
IMPACT OF QAT COMPONENTS

Configuration	FVD
No QAT (PTQ only)	187.3
+ Standard QAT	112.4
+ Learnable scales	102.1
+ Cross-modal attention QAT	99.8
+ Temporal smoothness loss	**99.2**

C. Kernel Performance

Table IX reports the per-layer runtime of our packed FP4 kernels against the baselines.

TABLE IX
KERNEL MICROBENCHMARKS (DIT-B ATTENTION LAYER)

Kernel	Time (us)
FP16 baseline	245
Naive FP4 (unpack to compute)	187
FP4 Triton (no packing)	132
FP4 Triton (packed)	89
FP4 FlashAttention (ours)	**67**

VIII. DISCUSSION

A. Why Does NVFP4-DiT Succeed Where Static FP4 Fails?

Static FP4 collapses because:

1) **Extreme outliers** in attention logits cause saturation (clip to ± 6 in scaled domain)
2) **Temporal variations** exceed block-wise scale range
3) **Cross-modal gradients** are mis-scaled for low precision

Our adaptive block-wise quantization dynamically adjusts block size per layer, while learnable scales in cross-attention optimize the dynamic range for audio-visual fusion. The temporal smoothness loss regularizes the model to produce coherence-respecting outputs.

B. Limitations

1) **Training complexity:** QAT adds 20% overhead vs. BF16 training (but total training is 66% shorter due to FP4 acceleration)
2) **Hardware requirement:** NVFP4 tensor cores only available on H100/B200 (not Ada/Ampere)
3) **Long video generation (over 128 frames):** Accumulated quantization error may degrade quality; we are exploring hierarchical quantization strategies
4) **Theoretical bounds are worst-case:** In practice, degradation is much smaller (under 1%)

C. Societal Impact

Positive: Democratizes video generation by reducing compute cost; enables on-device creative tools; lowers carbon footprint of generative AI.

Negative: Could lower the barrier for deepfake generation; responsible deployment should therefore include dataset governance, provenance mechanisms, and downstream content safeguards.

IX. Conclusion

This paper presented NVFP4-DiT, a framework for studying 4-bit training and inference for audio-guided video diffusion transformers. We developed a theoretical perspective on FP4 diffusion, introduced adaptive block-wise quantization and QAT for cross-modal attention, and described implementation choices for low-precision kernels and serving. The experimental results indicate that carefully designed 4-bit training can substantially reduce memory and computation while retaining competitive multimodal generation quality. These findings support further investigation into reliable low-precision training for video generation systems.

Additional implementation details, checkpoints, and qualitative examples are provided in the supplementary material.

Code Availability

Code, kernels, and related artifacts will be released upon publication.

References

[1] W. Peebles and S. Xie, "Scalable diffusion models with transformers," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2023.

[2] J. Ho, W. Chan, C. Saharia, J. Whang, R. Goyal, Y. Zhang, A. Srinivas, and T. Salimans, "Imagen Video: High-definition video generation with diffusion models," *arXiv preprint arXiv:2210.02303*, 2022.

[3] U. Singer, A. Polyak, T. Hayes, X. Yin, J. An, S. Zhang, Q. Hu, H. Yang, O. Ashual, O. Gafni *et al.*, "Make-A-Video: Text-to-video generation without text-video data," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[4] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022.

[5] OpenAI, "Sora: Creating video from text," *Technical report*, 2024.

[6] Stability AI, "Stable video diffusion," https://stability.ai/stable-video, 2023.

[7] J. Wang, H. Yuan, D. Chen, Y. Zhang, X. Wang, and S. Zhang, "ModelScope text-to-video technical report," *arXiv preprint arXiv:2308.06571*, 2023.

[8] L. Zhang, A. Gupta, and M. Chen, "Analyzing the computational costs of video diffusion models," in *Proc. Mach. Learn. Syst. (MLSys)*, 2024.

[9] A. Gupta, S. Reddy, and R. Kumar, "Efficient video generation: A survey," *IEEE Trans. Pattern Anal. Mach. Intell. (TPAMI)*, 2024.

[10] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2016.

[11] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," in *NIPS Deep Learning Workshop*, 2014.

[12] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018.

[13] NVIDIA, "NVFP4: 4-bit floating-point format for deep learning," *NVIDIA Developer Blog*, 2024.

[14] NVIDIA, "H100 Tensor Core GPU architecture," *White paper*, 2022.

[15] P. Tillet, H. T. Kung, and D. Cox, "Triton: An intermediate language and compiler for tiled neural network computations," in *Proc. Mach. Learn. Syst. (MLSys)*, 2019.

[16] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proc. ACM Symp. Oper. Syst. Princ. (SOSP)*, 2023.

[17] NVIDIA, "TensorRT-LLM: A TensorRT toolbox for large language models," https://github.com/NVIDIA/TensorRT-LLM, 2024.

[18] M. Bain, A. Nagrani, G. Varol, and A. Zisserman, "Frozen in time: A joint video and image encoder for end-to-end retrieval," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2021.

[19] H. Chen, W. Xie, A. Vedaldi, and A. Zisserman, "VGGSound: A large-scale audio-visual dataset," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process. (ICASSP)*, 2020.

[20] K. Soomro, A. R. Zamir, and M. Shah, "UCF101: A dataset of 101 human actions classes from videos in the wild," *arXiv preprint arXiv:1212.0402*, 2012.

[21] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020.

[22] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021.

[23] J. Ho, T. Salimans, A. Gritsenko, W. Chan, M. Norouzi, and D. J. Fleet, "Video diffusion models," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2022.

[24] R. Yang, P. Srivastava, and S. Mandt, "Latent video diffusion models for high-fidelity long video generation," *arXiv preprint arXiv:2211.13221*, 2022.

[25] X. Ma, Y. Wang, G. Jia, X. Chen, Z. Liu, Y. Li, C. Shen, and Y. Qiao, "Latte: Latent diffusion transformer for video generation," *arXiv preprint arXiv:2401.03048*, 2024.

[26] L. Zhu, D. Yang, T. Zhu, F. Reda, W. Chan, C. Saharia, M. Norouzi, and I. Kemelmacher-Shlizerman, "Tune-A-Video: One-shot tuning of image diffusion models for text-to-video generation," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2023.

[27] Y. Guo, C. Du, Z. Ma, and B. Chen, "Audio-guided video diffusion models," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024.

[28] Y. Bondarenko, M. Nagel, and T. Blankevoort, "Understanding and overcoming the challenges of quantizing diffusion models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[29] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and efficient post-training quantization for large language models," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2023.

[30] S. Esser, J. L. McKinstry, D. Bablani, R. Appuswamy, and D. S. Modha, "Learned step size quantization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2020.

[31] Z. Liu, D. S. Modha, and S. Esser, "LSQ++: Lower precision with higher accuracy," *arXiv preprint arXiv:2004.09576*, 2020.

[32] Y. Li, Y. Wang, and D. Xu, "EfficientDM: Efficient quantization-aware fine-tuning of diffusion models," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2023.

[33] NVIDIA, "FP8 formats for deep learning," *arXiv preprint arXiv:2209.05433*, 2022.

[34] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient fine-tuning of quantized LLMs," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2023.

[35] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[36] L. Zhao, Y. Zhang, and T. Chen, "FP4-CNN: 4-bit floating-point quantization for convolutional neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2023.

[37] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2022.

[38] J. S. Chung and A. Zisserman, "SyncNet: Audio-visual synchronization for video generation," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2019.

[39] T. Unterthiner, S. van Steenkiste, K. Kurach, R. Marinier, M. Michalski, and S. Gelly, "Fréchet Video Distance: A metric for video generation," in *NeurIPS Workshop*, 2019.

[40] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018.

Md Rakibul Islam Raihan He received the B.E. degree in Computer Science and Technology from Northwestern Polytechnical University (NWPU), Xi'an, China, in 2024, where he graduated as an Outstanding Undergraduate Graduate and ranked among the top 10 students in his cohort. He is currently pursuing the M.E. degree in Software Engineering at NWPU, where he ranks in the top 1% of his program. He is a Graduate Research Assistant with the Multimodal Learning and Video Generation Lab, NWPU. His research focuses on audio-visual fusion, self-supervised cross-modal representation learning, and text-to-video generation with Diffusion Transformer architectures for realistic video synthesis. He also served as a Technical Staff Intern at Infix.ai, where he designed NVFP4 pretraining pipelines for large language models and vision-language models and optimized inference with SGLang and Mooncake. His research interests include multimodal video generation, audio-visual fusion, deepfake detection based on visual, auditory, and temporal inconsistencies, and generative models, including diffusion models and GANs. He received the Chinese Government Scholarship and the Best Undergraduate Thesis Award.

Peng Zhang He received the B.E. degree from Xi'an Jiaotong University, Xi'an, China, in 2001, and the Ph.D. degree from Nanyang Technological University, Singapore, in 2011. He is currently a Full Professor with the School of Computer Science, Northwestern Polytechnical University, Xi'an, China. Prof. Zhang has published more than 100 research papers, including papers in CVPR, ACM Multimedia, IEEE Transactions on Image Processing, IEEE Transactions on Multimedia, and IEEE Transactions on Medical Imaging. He has served as a Program Chair for multiple IEEE international conferences. His current research interests include computer vision, pattern recognition, machine learning, and artificial intelligence. He has served as the Principal Investigator on more than 10 national grants and is also the Chief Scientist at Mekitec OY, Finland.